Encapsulation in C++

Encapsulation in C++..... 1
What is Encapsulation?..... 2
Real Life Example..... 2
Another Example..... 3
Why Encapsulation is Needed?..... 3
Getter and Setter Functions in C++..... 7
Solution: Getter and Setter Functions..... 8
Real-Life Example: Bank Account..... 14
Real-Life Example: Employee..... 15
Advantages of Getter and Setter Functions..... 16
Disadvantages..... 16
Summary..... 17
Encapsulation with Multiple Data Members..... 19
Drawback of Not Using Encapsulation..... 23
Encapsulation vs Data Hiding..... 24

What is Encapsulation?

Definition:

Encapsulation is an Object-Oriented Programming (OOP) concept in which **data (variables)** and the **functions (methods)** that operate on that data are combined into a single unit called a **class**.

It also **restricts direct access** to the internal data of the object by using **access specifiers**.

> **Simple Definition**
>
> Encapsulation means **wrapping data and functions together into a single unit (class)** and **protecting the data from unauthorized access.**

Real Life Example

Imagine an **ATM Machine**.

- Cash is stored inside the ATM.
- You cannot directly access the cash.
- You can only use
    - Insert Card
    - Enter PIN
    - Withdraw Money
    - Check Balance

Internally, the ATM performs many operations, but they are hidden from the user.

This is **Encapsulation**.



Another Example

Think of a **Car**.

You drive a car using

- Accelerator
- Brake
- Steering

You don't directly control

- Fuel Injection
- Engine Timing
- Gear Synchronization

Everything is encapsulated inside the car.

Why Encapsulation is Needed?

Without encapsulation,

- Anyone can change object data.
- Invalid values may be assigned.
- Program becomes difficult to maintain.
- Data becomes unsafe.

Example

```
class Student
{
public:
   int age;
};

int main()
{
   Student s;

   s.age = -20;    // Invalid
}
```

Here, negative age is allowed.

This is incorrect.

Encapsulation solves this problem.

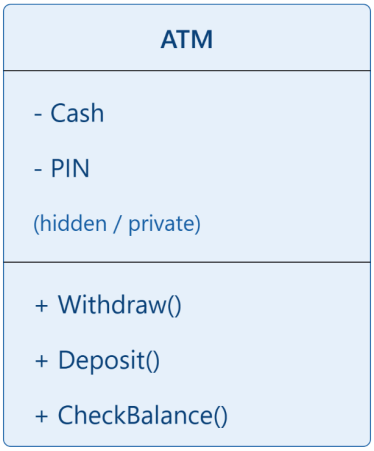
How Encapsulation is Achieved?

Encapsulation is achieved by

- Class
- Access Specifiers

Mostly,

- data → private
- functions → public



Access Specifiers

There are three access specifiers.

1) Private

Accessible only inside the class.

```
private:
  int age;
```

Cannot be accessed like

obj.age;

outside the class.

2) Public

Accessible everywhere.

public:
  void display();

3) Protected

Accessible

- inside class
- derived class

Used mainly in inheritance.

Access Table

Access Specifier	Same Class	Derived Class	Outside Class
Private	Yes	No	No
Protected	Yes	Yes	No
Public	Yes	Yes	Yes

Basic Encapsulation Example

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
private:
```

```
    // Private data member
    // It cannot be accessed directly from outside the class
    int rollNo;

public:
    // Member function to set the roll number
    void setRoll(int r)
    {
        // Assign the value of r to rollNo
        rollNo = r;
    }

    // Member function to display the roll number
    void getRoll()
    {
        cout << "Roll Number = "
             << rollNo;
    }
};

int main()
{
    // Create an object of Student class
    Student s;

    // Call setRoll() to assign 101 to rollNo
    s.setRoll(101);

    // Call getRoll() to display the roll number
    s.getRoll();

    return 0;  // Indicate successful program execution
}
```

Output
Roll Number = 101

Note: rollNo cannot be accessed directly. Only setRoll() and getRoll() can access it.

Using Getter and Setter Functions

Getter and Setter Functions in C++

Getter and Setter functions are **special member functions** used to **access (read)** and **modify (write)** the private data members of a class.

They are one of the most common ways to implement **encapsulation** in Object-Oriented Programming (OOP).

Why Do We Need Getter and Setter Functions?

Suppose we create a class like this:

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
private:
    // Private data member
    // It can be accessed only inside the Student class
    int age;
};

int main()
{
    // Create an object of Student class
    Student s;

    // Trying to access the private member directly
    // This will cause a compilation error
    s.age = 20;    // Error: age is private

    return 0;
}
```

Output

Error: 'age' is private within this context

Why?

Because `age` is declared as **private**, so it **cannot be accessed directly** from outside the class.

Private members can only be accessed by the class's own member functions.

Solution: Getter and Setter Functions

We create public functions that allow controlled access to the private data.

```
#include <iostream>
using namespace std;

// Define a class named Student
class Student
{
private:
    // Private data member
    // It cannot be accessed directly from outside the class
    int age;

public:
    // Member function to set the value of age
    void setAge(int a)
    {
        // Assign the parameter value to the private data member
        age = a;
    }

    // Member function to return the value of age
    int getAge()
    {
        return age;
    }
```

```
};

int main()
{
    // Create an object of Student class
    Student s;

    // Set the age using the public setter function
    s.setAge(20);

    // Get and display the age using the public getter function
    cout << "Age = " << s.getAge();

    return 0;  // Indicate successful program execution
}
```

Output : Age = 20

What is a Setter Function?

A **Setter** is a member function used to **assign or update** the value of a private data member.

Syntax

```
void setVariable(datatype value)
{
    variable = value;
}
```

Example

```
class Student
{
private:
    int age;

public:
    void setAge(int a)
    {
        age = a;
    }
};
```

Calling

Student s;

s.setAge(25);

stores

age = 25

inside the object.

What is a Getter Function?

A **Getter** is a member function used to **return (read)** the value of a private data member.

Syntax

```
class Student
{
private:
    int age;

public:
    void setAge(int a)
    {
        age = a;
    }
};
```

Example

```
class Student
{
private:
    int age;

public:
    int getAge()
    {
        return age;
    }
};
```

Calling

```
cout << s.getAge();
```

prints

25

Why Are They Used?

1. Data Hiding (Encapsulation)

Private variables cannot be accessed directly.

```
private:
   int salary;
```

Wrong

```
emp.salary = 50000;    // Error
```

Correct

```
emp.setSalary(50000);
```

2. Data Validation

The setter can check whether the entered value is valid before storing it.

Without validation

```
salary = -5000;
```

This is invalid.

Using setter

```
void setSalary(int s)
{
   if(s >= 0)
      salary = s;
   else
      cout << "Invalid Salary";
}
```

Now

emp.setSalary(-5000);

Output

Invalid Salary

The incorrect value is **not stored**.

3. Protect the Object

Suppose a bank account should never have a negative balance.

Setter

```
void setBalance(int b)
{
   if(b >= 0)
      balance = b;
}
```

Only valid values are stored.

4. Read-Only Variables

Sometimes we want the user to **read** a value but **not modify** it.

Example

```
class Student
{
private:
    int rollNo = 101;

public:
    int getRollNo()
    {
        return rollNo;
    }
};
```

Usage

cout << s.getRollNo();

Output

101

Since there is no `setRollNo()`, the roll number cannot be changed from outside the class.

5. Write-Only Variables

Sometimes data can be written but not read.

```
class Password
{
private:
   string password;

public:
   void setPassword(string p)
   {
      password = p;
   }
};
```

The user can set the password but cannot retrieve it using a getter.

Real-Life Example: Bank Account

```cpp
#include <iostream>
using namespace std;

class BankAccount
{
private:
    int balance;

public:
    void setBalance(int b)
    {
        if(b >= 0)
            balance = b;
        else
            cout << "Invalid Balance" << endl;
```

```
    }

    int getBalance()
    {
        return balance;
    }
};

int main()
{
    BankAccount account;

    account.setBalance(10000);

    cout << "Balance = " << account.getBalance() << endl;

    account.setBalance(-500);

    return 0;
}
```

Output

Balance = 10000
Invalid Balance

Real-Life Example: Employee

```
#include <iostream>
using namespace std;

class Employee
{
private:
    string name;
    int salary;

public:
    void setName(string n)
    {
```

```
        name = n;
    }

    void setSalary(int s)
    {
        if(s > 0)
            salary = s;
        else
            cout << "Invalid Salary" << endl;
    }

    string getName()
    {
        return name;
    }

    int getSalary()
    {
        return salary;
    }
};

int main()
{
    Employee emp;

    emp.setName("Rahul");
    emp.setSalary(45000);

    cout << "Name   : " << emp.getName() << endl;
    cout << "Salary : " << emp.getSalary() << endl;

    return 0;
}
```

Output

Name   : Rahul
Salary : 45000

Advantages of Getter and Setter Functions

- They implement **encapsulation** by hiding data members.
- They provide **controlled access** to private variables.
- They allow **validation** before updating data.
- They protect objects from invalid or inconsistent values.
- They make code easier to maintain because validation logic is centralized.
- They support **read-only** and **write-only** access when needed.

Disadvantages

- More code needs to be written compared to direct access.
- Very simple classes may end up with many small functions.
- Unnecessary getters and setters for every variable can reduce encapsulation if they simply expose all internal data without validation.

Summary

Getter	Setter
Used to read/access a private data member	Used to modify/update a private data member
Returns a value	Usually does not return a value (`void`)
No parameter is required (typically)	Takes one or more parameters
Example: `getAge()`	Example: `setAge(int age)`
Provides controlled read access	Provides controlled write access and validation

Encapsulation with Validation

One major advantage.

```
#include <iostream>
using namespace std;
```

```
class Student
{
private:
   int age;

public:

   void setAge(int a)
   {
       if(a>=0 && a<=120)
           age = a;
       else
           cout<<"Invalid Age";
   }

   int getAge()
   {
       return age;
   }
};

int main()
{
   Student s;

   s.setAge(-25);

   cout<<s.getAge();
}
```

Output

Invalid Age
0

Now invalid data cannot enter the object.

Encapsulation with Constructor

```
#include <iostream>
using namespace std;

class Student
{
private:
   int roll;

public:

   Student(int r)
   {
       roll = r;
   }

   void display()
   {
       cout<<roll;
   }
};

int main()
{
   Student s(10);

   s.display();
}
```

Encapsulation with Multiple Data Members

```
#include <iostream>
using namespace std;

class Student
{
private:
   int roll;
   string name;
   float marks;
```

```
public:

   void setData(int r,string n,float m)
   {
       roll = r;
       name = n;
       marks = m;
   }

   void display()
   {
       cout<<"Roll : "<<roll<<endl;
       cout<<"Name : "<<name<<endl;
       cout<<"Marks : "<<marks;
   }
};

int main()
{
   Student s;

   s.setData(1,"Rahul",89.5);

   s.display();
}
```

Output

Roll : 1
Name : Rahul
Marks : 89.5
]

Encapsulation with Bank Account Example

```
#include <iostream>
using namespace std;

class BankAccount
{
```

```
private:

   int balance;

public:

   BankAccount()
   {
       balance = 1000;
   }

   void deposit(int amount)
   {
       balance += amount;
   }

   void withdraw(int amount)
   {
       if(amount<=balance)
           balance -= amount;
       else
           cout<<"Insufficient Balance"<<endl;
   }

   void showBalance()
   {
       cout<<"Balance = "<<balance;
   }
};

int main()
{
   BankAccount b;

   b.deposit(500);

   b.withdraw(200);

   b.showBalance();
}
```

Output

Balance = 1300

Notice that

balance

is private.

Only

- deposit()
- withdraw()
- showBalance()

can change it.

Encapsulation with Private Helper Function

Private functions are also possible.

```
#include <iostream>
using namespace std;

class Rectangle
{
private:
   int length;
   int breadth;

   int area()
   {
       return length * breadth;
   }

public:

   void setData(int l,int b)
   {
       length=l;
       breadth=b;
   }
```

```
   void display()
   {
       cout<<"Area = "<<area();
   }
};

int main()
{
   Rectangle r;

   r.setData(10,5);

   r.display();
}
```

Output

Area = 50

The function

area()

cannot be called outside the class.

Drawback of Not Using Encapsulation

Without encapsulation

```
class Employee
{
public:

   int salary;
};
```

Anyone can write

Employee e;

e.salary = -50000;

Wrong value accepted.

With encapsulation

```
void setSalary(int s)
{
   if(s>=0)
       salary=s;
}
```

Now invalid salary is rejected.

Encapsulation vs Data Hiding

Many beginners think both are the same.

They are related but different.

Encapsulation	Data Hiding
Wrapping data and methods into a class	Restricting access to data
Achieved using classes	Achieved using private/protected members
Main OOP concept	A result of encapsulation

Relationship:

- **Encapsulation** is the mechanism (combining data and methods in a class).
- **Data Hiding** is one of the benefits achieved by encapsulation (preventing direct access